

DESIGN AND ANALYSIS OF ALGORITHMS

COMPREHENSIVE REPORT

Question 39 - Binary Search on Sorted Financial Transaction Data

Abstract

This report presents a comprehensive study of Binary Search in the context of a financial system that stores transaction IDs in sorted order. The report explains the working principle of Binary Search, the need for sorted data, implementation using Python, algorithm design, pseudocode, sample execution, complexity analysis, advantages, limitations, applications and detailed answers to all parts of the given question. This report is prepared according to the Design and Analysis of Algorithms rubric with emphasis on problem understanding, concept application, analysis, justification and solution design.

Introduction

Financial institutions perform millions of searches every day. Customers check transaction status, employees verify records, auditors inspect historical transactions and automated systems validate payments. Searching becomes inefficient if every record is examined one after another. Binary Search solves this problem by reducing the search interval by half after every comparison. This makes it one of the most efficient searching techniques for sorted data. However, its efficiency completely depends on maintaining sorted records.

Problem Statement

A financial system stores transaction IDs in sorted order to perform quick searching using Binary Search. Frequent updates insert new transaction IDs and disturb the sorted order. The objective is to explain Binary Search, analyse its performance when data is sorted and unsorted, discuss the importance of maintaining sorted order and implement the algorithm.

Objectives

The objectives are to understand Binary Search, implement it in Python, study its complexity, analyse sorted versus disturbed data, and justify why maintaining sorted order is necessary in financial databases.

Existing System

In an unsorted financial database, every transaction may need to be checked one by one using Linear Search. This increases response time significantly as the number of transactions grows. Although insertion is easy, searching becomes slow and inefficient.

Proposed System

The proposed system maintains transaction IDs in sorted order and performs searching using Binary Search. After every update, the data should be reordered if necessary so that Binary Search continues to work correctly.

Theory of Binary Search

Explanation 1: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Explanation 2: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Explanation 3: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Explanation 4: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the

sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Explanation 5: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Explanation 6: Binary Search follows the divide-and-conquer strategy. During every iteration, the algorithm compares the target transaction ID with the middle element of the sorted list. If the target is smaller, the right half is ignored; if greater, the left half is ignored. This repeated elimination of half the search space dramatically reduces the number of comparisons. In financial systems where millions of records are stored, this behaviour improves search efficiency and reduces execution time.

Algorithm

1. Read the number of transaction IDs.
2. Store all transaction IDs in sorted order.
3. Read the target transaction ID.
4. Set $low=0$ and $high=n-1$.
5. Repeat until low exceeds $high$.
6. Find middle index.
7. Compare middle value with target.
8. If equal return index.
9. If target is greater search right half.
10. Else search left half.
11. If loop ends return not found.

Pseudocode

```
BEGIN
Read n
Read sorted transaction IDs
Read target
 $low \leftarrow 0$ 
 $high \leftarrow n-1$ 
WHILE  $low \leq high$ 
```

```

mid ← (low+high)//2
IF A[mid]=target RETURN mid
ELSE IF A[mid]<target
    low ← mid+1
ELSE
    high ← mid-1
RETURN -1
END

```

Flowchart Explanation

Start → Read sorted transaction IDs → Read target → Initialise low and high → Calculate middle element → Compare target with middle → If equal display result → Otherwise move to left half or right half depending on comparison → Repeat until found or search interval becomes empty → Stop.

Start → Read sorted transaction IDs → Read target → Initialise low and high → Calculate middle element → Compare target with middle → If equal display result → Otherwise move to left half or right half depending on comparison → Repeat until found or search interval becomes empty → Stop.

Start → Read sorted transaction IDs → Read target → Initialise low and high → Calculate middle element → Compare target with middle → If equal display result → Otherwise move to left half or right half depending on comparison → Repeat until found or search interval becomes empty → Stop.

Start → Read sorted transaction IDs → Read target → Initialise low and high → Calculate middle element → Compare target with middle → If equal display result → Otherwise move to left half or right half depending on comparison → Repeat until found or search interval becomes empty → Stop.

Python Implementation

The following implementation uses the iterative version of Binary Search because it requires constant extra memory and is easy to understand.

```

def binary_search(arr,target):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==target:

```

```

        return mid
    elif arr[mid]<target:
        low=mid+1
    else:
        high=mid-1
    return -1

```

CODE

```

# ----- Input -----
# Design and Analysis of Algorithms
# Question 39
# Binary Search on Sorted Financial Transaction Data
# -----

def binary_search(transactions, target):

    low = 0
    high = len(transactions) - 1

    while low <= high:

        mid = (low + high) // 2

        if transactions[mid] == target:
            return mid

        elif transactions[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1

# ----- Search -----
position = binary_search(transactions, target)

# ----- Output -----

print("\nSorted Transaction List:")
print(transactions)

if position != -1:
    print("\nTransaction Found")
    print("Transaction ID :", target)
    print("Index :", position)
else:
    print("\nTransaction Not Found")

# ----- Complexity -----

print("\n----- Complexity Analysis -----")

print("Input Time Complexity      : O(n)")
print("Input Space Complexity      : O(n)")

print("Binary Search Best Case      : O(1)")
print("Binary Search Average Case   : O(log n)")
print("Binary Search Worst Case     : O(log n)")
print("Binary Search Space          : O(1)")

print("Output Time Complexity       : O(n)")
print("Output Space Complexity      : O(1)")

print("Overall Time Complexity      : O(n)")
print("Overall Space Complexity     : O(n)")

```

Program Explanation

Step 1: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the

comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 2: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 3: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 4: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 5: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 6: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 7: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Step 8: The program first reads the number of transactions and stores them in a sorted list. It then accepts the target transaction ID. The Binary Search function repeatedly calculates the middle index and compares it with the target. Depending on the

comparison, half of the remaining search space is discarded. This process continues until the transaction is found or all possibilities are exhausted.

Sample Execution

Input:

5

101

203

305

450

560

Target=450

Output:

Transaction Found

Index=3

Execution Trace:

Iteration 1: mid=2 value=305

Iteration 2: mid=3 value=450

Transaction Found.

OUTPUTS

```
Enter number of transactions: 5
Enter transaction IDs in Sorted Order:
1
2
3
4
5
Enter Transaction ID to search: 4
Sorted Transaction List:
[1, 2, 3, 4, 5]
Transaction Found
Transaction ID : 4
Index : 3
----- Complexity Analysis -----
Input Time Complexity      : O(n)
Input Space Complexity     : O(n)
Binary Search Best Case   : O(1)
Binary Search Average Case : O(log n)
Binary Search Worst Case  : O(log n)
Binary Search Space       : O(1)
Output Time Complexity     : O(n)
Output Space Complexity    : O(1)
Overall Time Complexity    : O(n)
Overall Space Complexity   : O(n)
```

Answer (a): Explain how Binary Search operates on sorted data

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Binary Search works only when data is sorted. The algorithm compares the target with the middle element, decides which half cannot contain the target, discards that half and continues searching only in the remaining half. Since the search space is halved after every comparison, the algorithm performs significantly fewer comparisons than Linear Search. This makes Binary Search highly suitable for financial transaction databases where fast retrieval is required.

Answer (b): Analyse its performance when order is maintained versus disturbed

When transaction records remain sorted, Binary Search achieves excellent performance. Best case occurs when the target is the middle element. Average and worst cases require logarithmic comparisons.

If updates disturb the sorted order, Binary Search cannot determine the correct half to continue searching. Consequently, it may fail even if the transaction exists.

Condition	Correctness	Time	Recommendation
Sorted	Correct	$O(\log n)$	Binary Search
Unsorted	Not Guaranteed	$O(n)$ using Linear Search	Sort then Binary Search

Maintaining sorted order enables Binary Search to eliminate half the search space repeatedly. Disturbing the order destroys this property, forcing the system to use slower methods or to sort the data again before searching.

Maintaining sorted order enables Binary Search to eliminate half the search space repeatedly. Disturbing the order destroys this property, forcing the system to use slower methods or to sort the data again before searching.

Maintaining sorted order enables Binary Search to eliminate half the search space repeatedly. Disturbing the order destroys this property, forcing the system to use slower methods or to sort the data again before searching.

Maintaining sorted order enables Binary Search to eliminate half the search space repeatedly. Disturbing the order destroys this property, forcing the system to use slower methods or to sort the data again before searching.

Answer (c): Importance of Maintaining Sorted Data

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and

scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Maintaining sorted data is essential because Binary Search depends entirely on ordering. Sorted data guarantees correctness, improves response time, reduces CPU usage during repeated searches, supports efficient financial reporting, enhances customer service and scales effectively as the database grows. Although updates require occasional re-sorting, the benefit gained during thousands of searches outweighs the maintenance cost.

Complexity Analysis

Operation	Time	Space
Input	$O(n)$	$O(n)$
Binary Search Best	$O(1)$	$O(1)$
Binary Search Average	$O(\log n)$	$O(1)$
Binary Search Worst	$O(\log n)$	$O(1)$
Output	$O(n)$	$O(1)$
Overall	$O(n)$	$O(n)$

Advantages

- Fast searching
- Fewer comparisons
- Low memory
- Suitable for large databases
- Reliable on sorted data
- Efficient for repeated searches

Limitations

- Requires sorted data
- Re-sorting after updates
- Not ideal for highly dynamic unsorted datasets

Applications

- Banking
- Online payments
- Library systems
- Hospital databases
- Inventory systems
- Student record management
- Payroll systems

Conclusion

Binary Search is one of the most powerful searching algorithms for sorted data. It reduces search time from linear growth to logarithmic growth by eliminating half of the remaining search space during every iteration. Maintaining sorted transaction records is therefore essential in financial systems where fast and accurate searching is required. The Python implementation demonstrates the practical application of Binary Search and the complexity analysis confirms its efficiency.

References

1. Introduction to Algorithms - Cormen et al.
2. Fundamentals of Computer Algorithms - Horowitz & Sahni.
3. Python Documentation.